```
begin
    using Symbolics
    using CairoMakie
    using Latexify      # lets Pluto render Symbolics expressions as LaTeX
end
```

Pure Bending of a Beam — Solving the Biharmonic Equation

Sadd, *Elasticity*, Example 8-2 — plane stress, straight beam of height $2c$, length $2l$ and unit thickness loaded by end moments M at $x = \pm l$ (Figure 8-2).

Steps:

1. Write the governing equation for the Airy stress function $\phi(x, y)$: the **biharmonic equation** $\nabla^4 \phi = 0$.
2. State the boundary conditions — and look carefully at *how the end conditions are relaxed* and what that costs us.
3. Pick a polynomial trial function, check the PDE, and enforce the boundary conditions symbolically with `Symbolics.jl`.
4. Recover the stress field, then strains (plane stress Hooke's law) and displacements (integrate the strain–displacement relations and fix rigid-body motion).
5. Compare with elementary Euler–Bernoulli beam theory.
6. Work a numerical example with `Makie.jl` plots, including a look at what happens at the beam ends.

1. Governing equation

With no body forces the equilibrium equations are satisfied identically by the Airy stress function ϕ:

$$\sigma_x = \frac{\partial^2 \phi}{\partial y^2}, \qquad \sigma_y = \frac{\partial^2 \phi}{\partial x^2}, \qquad \tau_{xy} = -\frac{\partial^2 \phi}{\partial x \partial y}$$

Compatibility of strains (plane stress, isotropic) then requires

$$\nabla^4 \phi = \frac{\partial^4 \phi}{\partial x^4} + 2\frac{\partial^4 \phi}{\partial x^2 \partial y^2} + \frac{\partial^4 \phi}{\partial y^4} = 0$$

2. Boundary conditions — and why the end conditions are relaxed

Top and bottom surfaces $y = \pm c$ are traction-free, and we impose this *pointwise* (at every x):

$$\sigma_y(x, \pm c) = 0, \qquad \tau_{xy}(x, \pm c) = 0$$

Ends $x = \pm l$. Figure 8-2 shows only a moment M. An ideal pointwise condition would prescribe the normal traction $\sigma_x(\pm l, y)$ for every y on the end face — but *we are not told how the moment is actually applied* (bearing plate? pair of bolts? welded end plate?). Sadd therefore writes only the **statically equivalent** (integral, or "weak") conditions, (8.1.15):

$$\tau_{xy}(\pm l, y) = 0, \qquad \int_{-c}^{c} \sigma_x(\pm l, y)\, dy = 0, \qquad \int_{-c}^{c} \sigma_x(\pm l, y)\, y\, dy = -M$$

i.e. *zero net axial force* and *net moment* M on each end. (The sign: with the usual beam convention a positive M puts the top fibre $y = +c$ in compression, so the resultant of $\sigma_x y$ is $-M$.)

Why relax?

1. **The real distribution is unknown or intractable.** A pointwise condition would require knowing the exact end traction; for most practical load introductions we don't, and even if we did, a polynomial ϕ could not match an arbitrary profile. Replacing the pointwise statement by two resultant conditions is what makes a closed-form polynomial solution possible at all.
2. **Saint-Venant's principle says it doesn't matter — away from the ends.** Sadd §5.6: *"the stress, strain and displacement fields caused by two different statically equivalent force distributions on parts of the body far away from the loading points are approximately the same."* Any two end loadings with the same force and moment resultants differ by a **self-equilibrated** load; the stresses produced by a self-equilibrated load on a strip of half-depth c decay roughly like $e^{-\lambda d/c}$ with distance d from the end, where λ is of order 2–4 (the leading Papkovich–Fadle eigenvalues; see the review by Horgan (1989) that Sadd cites). One beam depth ($d = 2c$) in from the end the disturbance is already down to a few percent or less.

What it costs

- **The solution is not unique near the ends.** Infinitely many exact elasticity solutions satisfy the relaxed conditions (8.1.15) — one for every traction profile with resultants $(0, -M)$. Our polynomial solution is simply the one whose end traction happens to be *linear in* y. All of them agree in the interior; they differ in a boundary layer of width $\sim 2c$ at each end.
- **So the solution is "not necessarily valid near the ends of the beam"** (Sadd). The stresses the solution *reports* on $x = \pm l$ are just the linear profile $\sigma_x = -My/I$; the real stresses there depend on how the moment is actually introduced.
- **Pure bending is a lucky case.** The linear traction $\sigma_x(\pm l, y) = -My/I$ is a perfectly reasonable way to apply a moment. If the moment *is* applied that way, the relaxed solution satisfies the pointwise end conditions too and is exact everywhere. For other loadings (Example 8-3 onward) the end traction implied by the solution is usually *not* realisable, and the end region genuinely differs.

The same relaxation also appears in the displacement "fixity" conditions below: pinning the points $(\pm l, 0)$ idealises a real support, which again only matters near the supports.

3. Symbolic setup

Define the independent variables, the problem parameters, and the differential operators.

```
Differential(y, 1)
```

```
begin
    @variables x y
    @variables M l c E ν          # end moment (per unit thickness), half-length, half-height, Young's modulus, Poisson's ratio
    Dx = Differential(x)
    Dy = Differential(y)
end
```

⚠ Attempting to print a symbolic expression in a Pluto notebook. Please run `import Latexify` to enable pretty-printing of symbolic expressions. This warning will only display once.

biharmonic

```
biharmonic(ϕ)
```

Apply the biharmonic operator $\nabla^4 = \partial^4/\partial x^4 + 2\,\partial^4/\partial x^2\partial y^2 + \partial^4/\partial y^4$ to a symbolic expression.

```
"""
    biharmonic(ϕ)

Apply the biharmonic operator ∇⁴ = ∂⁴/∂x⁴ + 2 ∂⁴/∂x²∂y² + ∂⁴/∂y⁴ to a symbolic expression.
"""
biharmonic(ϕ) = expand_derivatives(Dx(Dx(Dx(Dx(ϕ)))) + 2*Dx(Dx(Dy(Dy(ϕ)))) + Dy(Dy(Dy(Dy(ϕ)))))
```

stresses

```
stresses(ϕ) -> (σx, σy, τxy)
```

Stresses from the Airy stress function.

```
"""
    stresses(ϕ) -> (σx, σy, τxy)

Stresses from the Airy stress function.
"""
stresses(ϕ) = (expand_derivatives(Dy(Dy(ϕ))),
               expand_derivatives(Dx(Dx(ϕ))),
               expand_derivatives(-Dx(Dy(ϕ))))
```

Two small helpers for polynomials, which is all we need here:

- `∫(expr, v)` — antiderivative of a polynomial in `v` (term by term, $v^n \to v^{n+1}/(n+1)$);
- `coeffs_in(expr, v)` — the coefficients of a polynomial in `v`. A condition that must hold **for all** `v` (e.g. $\sigma_y(x, c) = 0$ for every x) is equivalent to *every coefficient* being zero.

```
coeffs_in (generic function with 1 method)
```

```
begin
    function ∫(expr, v)
        d, r = Symbolics.polynomial_coeffs(expand(expr), [v])
        iszero(Symbolics.value(r)) || error("expression is not a polynomial in $v")
        s = 0
        for (mono, co) in d
            n = Symbolics.degree(mono, v)
            s += co * v^(n + 1) / (n + 1)
        end
        return expand(s)
    end

    ∫(expr, v, a, b) = expand(substitute(∫(expr, v), Dict(v => b)) - substitute(∫(expr, v), Dict(v => a)))

    coeffs_in(expr, v) = collect(values(first(Symbolics.polynomial_coeffs(expand(expr), [v]))))
end
```

4. Trial stress function and solution of the PDE

The end conditions are moment resultants, so the end traction must vary with y; the simplest possibility is a **linear** σ_x, which comes from a **cubic** ϕ. Rather than guess Sadd's $\phi = A_{03}y^3$ outright, take the full polynomial of degree ≤ 3 (linear and constant terms produce no stress and are dropped) and let the boundary conditions eliminate what they must:

$$\phi = A_{20}x^2 + A_{11}xy + A_{02}y^2 + A_{30}x^3 + A_{21}x^2y + A_{12}xy^2 + A_{03}y^3$$

$$y^2\,\mathtt{A02} + \mathtt{A11}\,x\,y + x^2\,\mathtt{A20} + y^3\,\mathtt{A03} + y^2\,\mathtt{A12}\,x + x^2\,\mathtt{A21}\,y + x^3\,\mathtt{A30}$$

```
begin
    @variables A20 A11 A02 A30 A21 A12 A03
    unknowns = [A20, A11, A02, A30, A21, A12, A03]
    ϕ_trial = A20*x^2 + A11*x*y + A02*y^2 + A30*x^3 + A21*x^2*y + A12*x*y^2 + A03*y^3
end
```

Biharmonic check — every term has degree ≤ 3, so all 4th derivatives vanish and the PDE is satisfied for *any* coefficients:

```
∇⁴ϕ =
```

$$0$$

```
1 ∇⁴ϕ = biharmonic(ϕ_trial)
```

Stresses from the trial function (linear in x and y):

▸ $(2\,\mathtt{A02} + 6\,\mathtt{A03}\,y + 2\,\mathtt{A12}\,x,\ 2\,\mathtt{A20} + 2\,\mathtt{A21}\,y + 6\,\mathtt{A30}\,x,\ -\mathtt{A11} - 2\,\mathtt{A12}\,y - 2\,\mathtt{A21}\,x)$

```
1 σx_trial, σy_trial, τxy_trial = stresses(ϕ_trial)
```

Boundary conditions. The pointwise conditions on $y = \pm c$ must hold for all x, and the shear-free end condition for all y, so we match polynomial coefficients. The two end conditions are integrals over the depth. Collecting everything gives a list of linear equations in the seven coefficients:

```
bcs =
```

▸ $[2 \cdot \mathtt{A20} + 2 \cdot \mathtt{A21} \cdot c = 0,\ 6 \cdot \mathtt{A30} = 0,\ 2 \cdot \mathtt{A20} - 2 \cdot \mathtt{A21} \cdot c = 0,\ 6 \cdot \mathtt{A30} = 0,\ -\mathtt{A11} - 2 \cdot \mathtt{A12} \cdot c = 0,\ -2 \cdot \mathtt{A21}$

```
1  bcs = vcat(
2      # pointwise, for all x:  σy(x, ±c) = 0
3      [e ~ 0 for e in coeffs_in(substitute(σy_trial,  Dict(y =>  c)), x)],
4      [e ~ 0 for e in coeffs_in(substitute(σy_trial,  Dict(y => -c)), x)],
5      # pointwise, for all x:  τxy(x, ±c) = 0
6      [e ~ 0 for e in coeffs_in(substitute(τxy_trial, Dict(y =>  c)), x)],
7      [e ~ 0 for e in coeffs_in(substitute(τxy_trial, Dict(y => -c)), x)],
8      # pointwise, for all y:  τxy(±l, y) = 0
9      [e ~ 0 for e in coeffs_in(substitute(τxy_trial, Dict(x =>  l)), y)],
10     [e ~ 0 for e in coeffs_in(substitute(τxy_trial, Dict(x => -l)), y)],
11     # relaxed (resultant) end conditions:  ∫σx dy = 0,  ∫σx y dy = -M   at x = ±l
12     [∫(substitute(σx_trial, Dict(x =>  l)),     y, -c, c) ~ 0],
13     [∫(substitute(σx_trial, Dict(x => -l)),     y, -c, c) ~ 0],
14     [∫(substitute(σx_trial, Dict(x =>  l)) * y, y, -c, c) ~ -M],
15     [∫(substitute(σx_trial, Dict(x => -l)) * y, y, -c, c) ~ -M],
16 )
```

There are more equations than unknowns (several are duplicates, e.g. $6A_{30} = 0$ appears twice), but the system is consistent. A tidy way to solve a consistent overdetermined linear system $\mathbf{A}\mathbf{a} = \mathbf{b}$ exactly is through the normal equations $\mathbf{A}^T\mathbf{A}\,\mathbf{a} = \mathbf{A}^T\mathbf{b}$, which are square:

coeffs = $[0,\ 0,\ 0,\ 0,\ 0,\ 0,\ \frac{-M}{4\,c^3}]$

```
coeffs = let
    lhs = [eq.lhs - eq.rhs for eq in bcs]            # every entry must equal zero
    A   = Symbolics.jacobian(lhs, unknowns)           # linear in the unknowns ⇒ A is constant
    b   = -substitute.(lhs, Ref(Dict(unknowns .=> 0)))
    AtA = simplify.(A' * A)
    Atb = simplify.(A' * b)
    simplify.(Symbolics.symbolic_linear_solve([(AtA * unknowns)[i] ~ Atb[i] for i in eachindex(unknowns)], unknowns))
end
```

coeff_map = Dict(A02 A20 A11 A21 A30 A12 A03)

$$\Rightarrow 0,\ \Rightarrow 0,\ \Rightarrow 0,\ \Rightarrow 0,\ \Rightarrow 0,\ \Rightarrow 0,\ \Rightarrow \frac{-M}{4\,c^3}$$

```
coeff_map = Dict(unknowns .=> coeffs)
```

Every coefficient vanishes except $A_{03} = -M/(4c^3)$, exactly Sadd's choice (8.1.16):

ϕ =

$$\frac{-y^3\ M}{4\,c^3}$$

```
ϕ = simplify(substitute(ϕ_trial, coeff_map))
```

5. Stress field

$(\frac{-3\ M\ y}{2\,c^3},\ 0,\ 0)$

```
σx, σy, τxy = stresses(ϕ)
```

$$\sigma_x = -\frac{3M}{2c^3}\,y, \qquad \sigma_y = 0, \qquad \tau_{xy} = 0$$

Equation (8.1.18). Verify **all** boundary conditions — pointwise on the long edges, and in the resultant sense on the ends (each residual should be zero):

```
bc_residuals = ▸[0, 0, 0, 0, 0, 0, 0, 0]
```

```
bc_residuals = simplify.([
    substitute(σy,  Dict(y =>  c)),
    substitute(σy,  Dict(y => -c)),
    substitute(τxy, Dict(y =>  c)),
    substitute(τxy, Dict(y => -c)),
    substitute(τxy, Dict(x =>  l)),
    substitute(τxy, Dict(x => -l)),
    ∫(substitute(σx, Dict(x => l)),     y, -c, c),          # net axial force on the end
    ∫(substitute(σx, Dict(x => l)) * y, y, -c, c) + M,      # net moment on the end
])
```

Note what the solution says *on* the end face: $\sigma_x(\pm l, y) = -3My/(2c^3)$, a linear traction. The boundary conditions never asked for this profile — only for its resultants. Any other profile with the same resultants would be an equally valid end loading, and the solution could not tell the difference (Section 9).

6. Strains and displacements

Plane stress Hooke's law ($\sigma_z = 0$):

$$\varepsilon_x = \frac{1}{E}(\sigma_x - \nu\sigma_y), \quad \varepsilon_y = \frac{1}{E}(\sigma_y - \nu\sigma_x), \quad \gamma_{xy} = \frac{2(1+\nu)}{E}\tau_{xy}$$

▸$\left(\frac{-3\,M\,y}{2\,c^3\,E},\ \frac{3\,M\,y\,\nu}{2\,c^3\,E},\ 0\right)$

```
begin
    εx  = simplify((σx - ν*σy)/E)
    εy  = simplify((σy - ν*σx)/E)
    γxy = simplify(2(1 + ν)/E * τxy)
    (εx, εy, γxy)
end
```

Strain–displacement relations:

$$\varepsilon_x = \frac{\partial u}{\partial x}, \quad \varepsilon_y = \frac{\partial v}{\partial y}, \quad \gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}$$

Integrate the first two, (8.1.19):

$$u = \int \varepsilon_x\,dx + f(y) = -\frac{3M}{2Ec^3}xy + f(y), \qquad v = \int \varepsilon_y\,dy + g(x) = \frac{3\nu M}{4Ec^3}y^2 + g(x)$$

The shear relation with $\gamma_{xy} = 0$ gives (8.1.20)

$$-\frac{3M}{2Ec^3}x + f'(y) + g'(x) = 0$$

The only way a function of x alone can equal a function of y alone is for both to be the same constant, call it ω_0: $f'(y) = \omega_0$ and $g'(x) = \frac{3M}{2Ec^3}x - \omega_0$. Integrating, (8.1.21):

$$f(y) = \omega_0 y + u_0, \qquad g(x) = \frac{3M}{4Ec^3}x^2 - \omega_0 x + v_0$$

u_0, v_0, ω_0 are **rigid-body** translation and rotation — they produce no strain and are not determined by the stresses. They are fixed by displacement *fixity* conditions chosen to match the physical support. Sadd takes the beam as simply supported at its ends: $v(\pm l, 0) = 0$ and $u(-l, 0) = 0$.

▸ $\left(\mathrm{u0} + \frac{-3\,M\,x\,y}{2\,c^3\,E} + y\,\omega 0,\ \mathrm{v0} + \frac{3\,x^2\,M}{4\,c^3\,E} + \frac{3\,y^2\,M\,\nu}{4\,c^3\,E} - x\,\omega 0\right)$

```
begin
    @variables u0 v0 ω0
    u_part = ∫(εx, x)                    # particular part from ∂u/∂x = εx
    v_part = ∫(εy, y)                    # particular part from ∂v/∂y = εy
    # shear relation: f'(y) + g'(x) = -(∂u_part/∂y + ∂v_part/∂x); the right side is a
    function of x only here
    rhs_shear = expand(-expand_derivatives(Dy(u_part) + Dx(v_part)))
    f_y = ω0*y + u0
    g_x = ∫(rhs_shear, x) - ω0*x + v0
    u_general = u_part + f_y
    v_general = v_part + g_x
    (u_general, v_general)
end
```

Fixity conditions — simply supported: $v(\pm l, 0) = 0$, $u(-l, 0) = 0$. Solve for the rigid-body constants:

```
rigid_body = ▸Dict(  ω0          v0          u0  )
```

$$\Rightarrow 0, \quad \Rightarrow \frac{-3\,l^2\,M}{4\,c^3\,E}, \quad \Rightarrow 0$$

```
rigid_body = let
    fixity = [
        substitute(v_general, Dict(x =>  l, y => 0)) ~ 0,
        substitute(v_general, Dict(x => -l, y => 0)) ~ 0,
        substitute(u_general, Dict(x => -l, y => 0)) ~ 0,
    ]
    sol = simplify.(Symbolics.symbolic_linear_solve(fixity, [u0, v0, ω0]))
    Dict([u0, v0, ω0] .=> sol)
end
```

$$\left(\frac{-3\,M\,x\,y}{2\,c^3\,E},\ \frac{-3\,l^2\,M+3\,x^2\,M+3\,y^2\,M\,\nu}{4\,c^3\,E}\right)$$

```
begin
    u = simplify(substitute(u_general, rigid_body))
    v = simplify(substitute(v_general, rigid_body))
    (u, v)
end
```

So $u_0 = \omega_0 = 0$ and $v_0 = -3Ml^2/(4Ec^3)$:

$$u = -\frac{3M}{2Ec^3}\,xy, \qquad v = \frac{3M}{4Ec^3}\left[\nu y^2 + x^2 - l^2\right]$$

Check that the displacement field reproduces all three strains:

```
strain_check = [0, 0, 0]
```

```
strain_check = simplify.([
    expand_derivatives(Dx(u)) - εx,
    expand_derivatives(Dy(v)) - εy,
    expand_derivatives(Dy(u) + Dx(v)) - γxy,
])
```

7. Comparison with Euler–Bernoulli beam theory

With unit thickness the second moment of area is $I = \int_{-c}^{c} y^2\,dy = 2c^3/3$, so the elasticity solution reads (8.1.22)

$$\sigma_x = -\frac{M}{I}y, \qquad u = -\frac{Mxy}{EI}, \qquad v = \frac{M}{2EI}\left[\nu y^2 + x^2 - l^2\right]$$

Elementary (Euler–Bernoulli) theory assumes plane sections remain plane and normal to the axis, $\sigma_y = 0$, and gives (8.1.23)

$$\sigma_x^{EB} = -\frac{M}{I}y, \qquad v^{EB}(x) = \frac{M}{2EI}\left[x^2 - l^2\right], \qquad u^{EB} = -y\,\frac{dv^{EB}}{dx} = -\frac{Mxy}{EI}$$

Form the differences symbolically:

▸ (I = $\frac{2}{3}\,c^3$, σx_EB = $\frac{-3\,M\,y}{2\,c^3}$, v_EB = $\frac{3\,M\,\left(-l^2+x^2\right)}{4\,c^3\,E}$, u_EB = $\frac{-3\,M\,x\,y}{2\,c^3\,E}$)

```
begin
    I_sec = ∫(y^2, y, -c, c)                # = 2c³/3 (unit thickness)
    σx_EB = -M*y/I_sec
    v_EB  = M/(2E*I_sec) * (x^2 - l^2)
    u_EB  = expand_derivatives(-y * Dx(v_EB))
    (I = I_sec, σx_EB = σx_EB, v_EB = v_EB, u_EB = u_EB)
end
```

EB_differences = ▸ (Δσx = 0, Δu = 0, Δv = $\frac{3\,y^2\,M\,\nu}{4\,c^3\,E}$)

```
EB_differences = (
    Δσx = simplify(σx - σx_EB),
    Δu  = simplify(u - u_EB),
    Δv  = simplify(v - v_EB),
)
```

Bending stress and axial displacement are identical. Pure bending is the one case where the Euler–Bernoulli kinematic assumption (plane sections remain plane) is *exactly* satisfied by the elasticity solution — the elasticity u is linear in y. The only difference is in v:

$$v - v^{EB} = \frac{\nu M}{2EI}\,y^2$$

Beam theory tracks the centreline only, $v(x,0)$, where the two agree exactly. Off the centreline the elasticity v picks up a Poisson term: the fibres in compression (top) thicken and those in tension (bottom) thin, so a cross-section does not translate rigidly — this is the **anticlastic** curvature of the cross-section that beam theory ignores. The next example in Sadd (uniform transverse load, 8-3) is where the bending stress itself starts to differ.

Why "plane sections remain plane" is *demonstrated* here, not assumed

The elasticity solution never assumes anything about how cross-sections deform. It only requires equilibrium, compatibility and Hooke's law at every point, plus the boundary conditions. Whatever shape the cross-sections take is therefore an **output** of the solution. In this problem that output happens to coincide with the Euler–Bernoulli kinematic assumption, which is why Sadd says the classic assumption has been *verified* rather than imposed.

Where it shows up. The axial displacement that came out of integrating the strains is

$$u(x,y) = -\frac{M}{EI}\,xy$$

Fix a cross-section at the station $x = x_0$. The material points that were on the vertical line $x = x_0$ move to

$$x_{\text{new}} = x_0 + u(x_0, y) = x_0 - \frac{M x_0}{EI}\, y$$

This is **linear in y**. A straight line mapped by a linear function of y is still a straight line, so the cross-section — a plane, i.e. a line in 2-D — is still a plane after deformation. Its slope relative to the vertical is $-Mx_0/(EI)$: the section has rotated rigidly through the small angle $\theta(x_0) = Mx_0/(EI)$ about the neutral axis, but it has not bowed, warped or kinked. In the deformed-shape plot above, the red vertical grid lines stay straight and simply tilt.

Why it is not a coincidence. Two features of the solution force this:

- $\tau_{xy} \equiv 0$, so $\gamma_{xy} = \partial u/\partial y + \partial v/\partial x = 0$ everywhere. With no shear distortion, a line that starts perpendicular to the beam axis stays perpendicular to the deformed axis — the *"and remain normal to the axis"* half of the Euler–Bernoulli assumption.
- $\sigma_x = -My/I$ is linear in y and independent of x. Through Hooke's law $\varepsilon_x = \partial u/\partial x$ is linear in y, and integrating in x keeps it linear in y. Linear u in y is exactly *"plane sections remain plane"*.

A check by hand. If sections stay normal to the axis, the rotation of the section at x_0 must equal the slope of the deflected centreline there. From the solution $v(x, 0) = \frac{M}{2EI}(x^2 - l^2)$, so $dv/dx = Mx/(EI)$, which is the section rotation $\theta(x_0)$ found above. The two agree exactly, so $u = -y\, dv/dx$ holds — this *is* the Euler–Bernoulli displacement field, and the symbolic result $\Delta u = 0$ in `EB_differences` is its confirmation.

Why Sadd warns it will not hold in general. For the uniformly loaded beam of Example 8-3, τ_{xy} is no longer zero, so $\gamma_{xy} \neq 0$ and lines normal to the axis shear over. The stress function also gives σ_x a y^3 term, so u picks up a y^3 contribution and sections **warp**. Pure bending is the special case in which the solution contains only the linear term — precisely the case in which elementary beam theory is exact.

8. Numerical example

A steel beam, $2l = 2000$ mm long and $2c = 200$ mm deep (unit thickness, so M is a moment *per mm of width*). Take $M = 1{,}000$ kN·mm per mm — chosen so the extreme-fibre stress is a round number.

```
params = ▸(M = 1.0e6, l = 1000.0, c = 100.0, E = 200000.0, ν = 0.3)
```

```
params = (M = 1.0e6,        # N·mm per mm thickness
          l = 1000.0,       # mm  (half-length)
          c = 100.0,        # mm  (half-height)
          E = 200_000.0,    # MPa
          ν = 0.3)
```

```
RuntimeGeneratedFunction(#=in Symbolics=#, #=using Symbolics=#, :((x, y, M, l, c, E, ν)->#= 
            #= /Users/crismoen/.julia/packages/Symbolics/JBOvO/src/build_function.jl:198 =#
            begin
                (/)((*)((*)(3, M), (+)((*)(-1, (^)(l, 2)), (^)(x, 2))), (*)((*)(4, E), (^)
            end
        end)))
```

```
begin
    # build_function compiles each symbolic expression to a callable f(x, y, M, l, c, E, ν)
    args = (x, y, M, l, c, E, ν)
    σx_f    = build_function(σx,    args...; expression = Val(false))
    σy_f    = build_function(σy,    args...; expression = Val(false))
    τxy_f   = build_function(τxy,   args...; expression = Val(false))
    u_f     = build_function(u,     args...; expression = Val(false))
    v_f     = build_function(v,     args...; expression = Val(false))
    σx_EB_f = build_function(σx_EB, args...; expression = Val(false))
    v_EB_f  = build_function(v_EB,  args...; expression = Val(false))
end
```

▸ (I = 6.66667e5, σx = (-150.0, 150.0), σy = (0, 0), τxy = (0, 0), u = (-0.75, 0.75), v = (-3.75,

```
begin
    p = params
    xs = range(-p.l, p.l, length = 81)
    ys = range(-p.c, p.c, length = 21)
    ev(f) = [f(xi, yi, p.M, p.l, p.c, p.E, p.ν) for xi in xs, yi in ys]
    Σx, Σy, Txy = ev(σx_f), ev(σy_f), ev(τxy_f)
    U, V = ev(u_f), ev(v_f)
    I_num = 2p.c^3/3
    (I = I_num, σx = extrema(Σx), σy = extrema(Σy), τxy = extrema(Txy), u = extrema(U), v = extrema(V))
end
```

Elasticity vs Euler–Bernoulli, side by side

Evaluate both theories at the same points: the extreme fibres at midspan and the midspan deflection.

comparison =

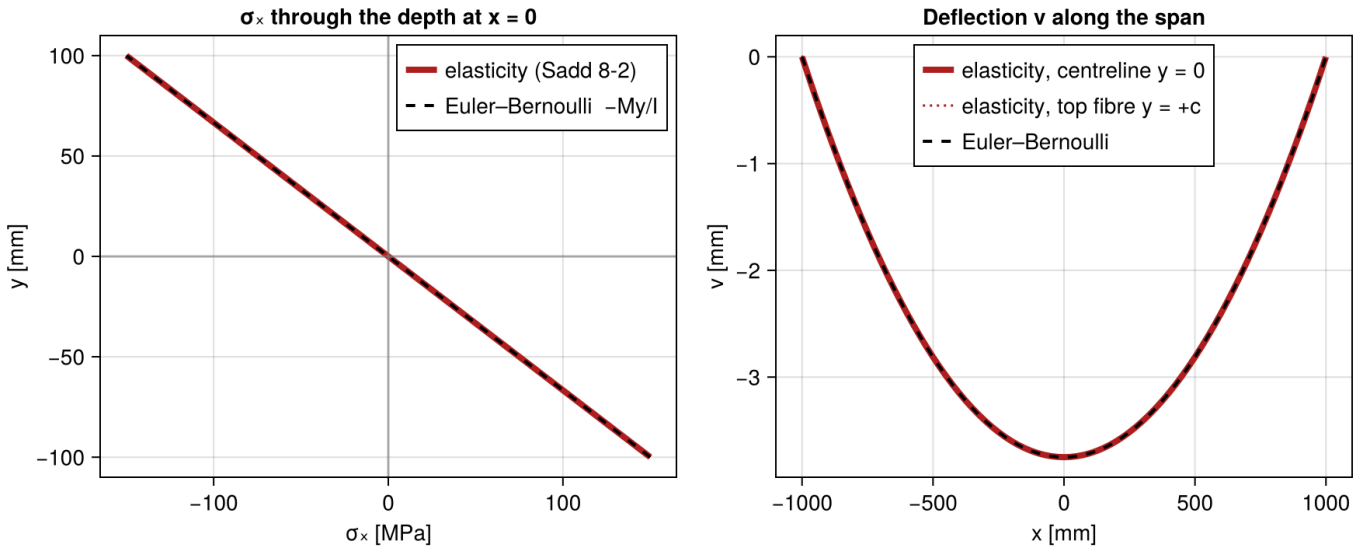
quantity	elasticity (Sadd 8-2)	Euler–Bernoulli	difference
σₓ top fibre (0, +c) [MPa]	-150.0	-150.0	0.0
σₓ bottom fibre (0, −c) [MPa]	150.0	150.0	0.0
σₓ at end face (l, +c) [MPa]	-150.0	-150.0	0.0
v at midspan centreline (0, 0) [mm]	-3.75	-3.75	0.0
v at midspan top fibre (0, +c) [mm]	-3.7388	-3.75	0.0112
v at midspan bottom fibre (0, −c) [mm]	-3.7388	-3.75	0.0112

```
comparison = let
    f(g, xi, yi) = g(xi, yi, p.M, p.l, p.c, p.E, p.ν)
    rows = [
        ("σₓ top fibre (0, +c)  [MPa]",          f(σx_f, 0.0,  p.c), f(σx_EB_f, 0.0,  p.c)),
        ("σₓ bottom fibre (0, -c)  [MPa]",       f(σx_f, 0.0, -p.c), f(σx_EB_f, 0.0, -p.c)),
        ("σₓ at end face (l, +c)  [MPa]",        f(σx_f, p.l,  p.c), f(σx_EB_f, p.l,  p.c)),
        ("v at midspan centreline (0, 0)  [mm]", f(v_f, 0.0, 0.0),   f(v_EB_f, 0.0, 0.0)),
        ("v at midspan top fibre (0, +c)  [mm]", f(v_f, 0.0,  p.c),  f(v_EB_f, 0.0,  p.c)),
        ("v at midspan bottom fibre (0, -c) [mm]", f(v_f, 0.0, -p.c), f(v_EB_f, 0.0, -p.c)),
    ]
    hdr = "| quantity | elasticity (Sadd 8-2) | Euler-Bernoulli | difference |\n|---|---|---|---|\n"
    body = join(["| $(r[1]) | $(round(r[2], digits = 4)) | $(round(r[3], digits = 4)) | $(round(r[2] - r[3], digits = 4)) |" for r in rows], "\n")
    Markdown.parse(hdr * body)
end
```

The bending stresses agree to machine precision everywhere, including on the end face (where both theories are only reporting the *assumed* linear end traction). The centreline deflections agree. The only discrepancy is the Poisson term $\nu M c^2/(2EI)$ at the extreme fibres — here a few hundredths of a millimetre on a 3.75 mm deflection, i.e. $\nu c^2/l^2 = 0.3\%$ of the centreline value. For a slender beam ($c \ll l$) beam theory is essentially exact in pure bending.

Bending stress through the depth and deflection along the span

```
fig_compare =
```




```julia
fig_compare = let
    fig = Figure(size = (900, 380))

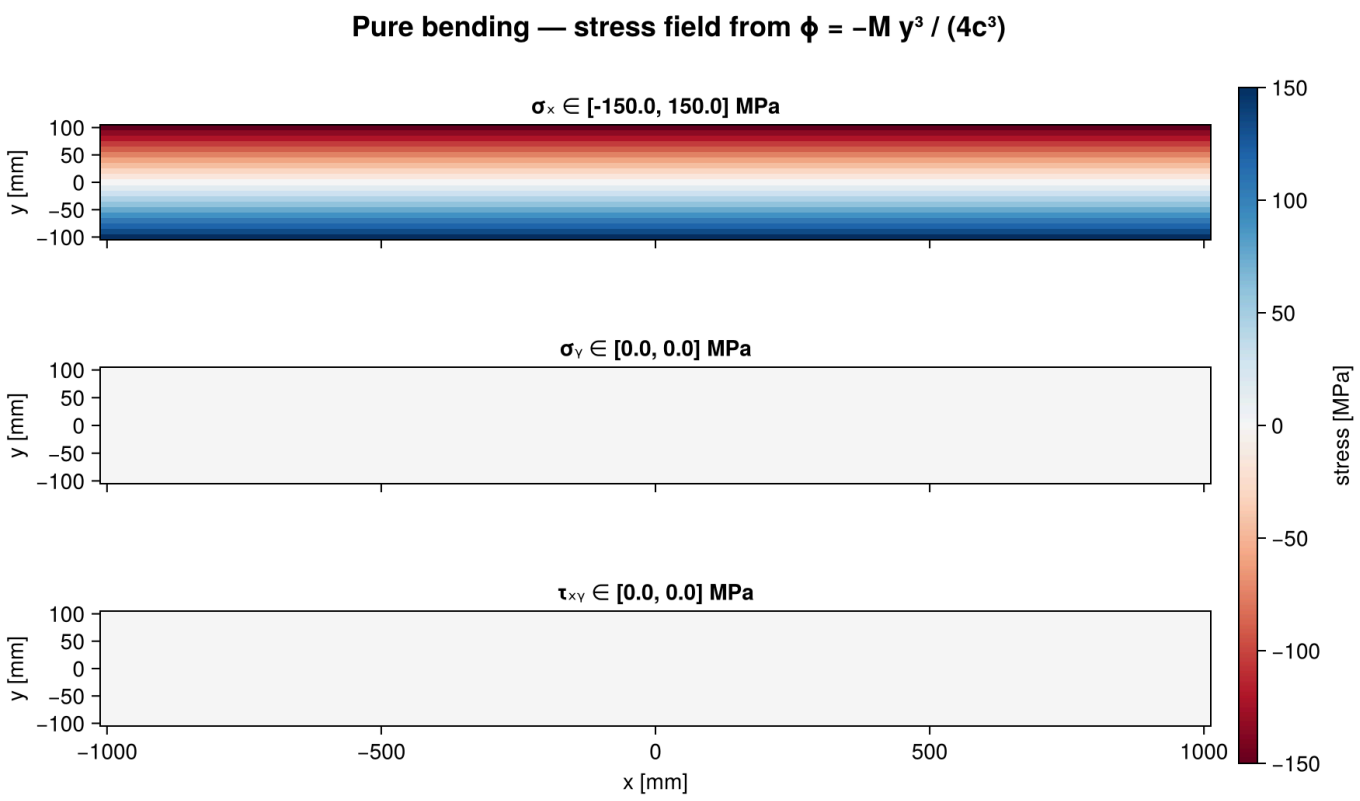
    # σx through the depth at midspan
    ax1 = Axis(fig[1, 1], title = "σₓ through the depth at x = 0",
               xlabel = "σₓ [MPa]", ylabel = "y [mm]")
    yy = range(-p.c, p.c, length = 41)
    lines!(ax1, [σx_f(0.0, yi, p.M, p.I, p.c, p.E, p.ν) for yi in yy], yy,
          color = :firebrick, linewidth = 4, label = "elasticity (Sadd 8-2)")
    lines!(ax1, [σx_EB_f(0.0, yi, p.M, p.I, p.c, p.E, p.ν) for yi in yy], yy,
          color = :black, linewidth = 2, linestyle = :dash, label = "Euler–Bernoulli −My/I")
    vlines!(ax1, [0.0], color = (:gray, 0.6))
    hlines!(ax1, [0.0], color = (:gray, 0.6))
    axislegend(ax1, position = :rt)

    # deflection along the span
    ax2 = Axis(fig[1, 2], title = "Deflection v along the span",
               xlabel = "x [mm]", ylabel = "v [mm]")
    lines!(ax2, xs, [v_f(xi, 0.0, p.M, p.I, p.c, p.E, p.ν) for xi in xs],
          color = :firebrick, linewidth = 4, label = "elasticity, centreline y = 0")
    lines!(ax2, xs, [v_f(xi, p.c, p.M, p.I, p.c, p.E, p.ν) for xi in xs],
          color = :firebrick, linewidth = 1.5, linestyle = :dot, label = "elasticity, top fibre y = +c")
    lines!(ax2, xs, [v_EB_f(xi, 0.0, p.M, p.I, p.c, p.E, p.ν) for xi in xs],
          color = :black, linewidth = 2, linestyle = :dash, label = "Euler–Bernoulli")
    axislegend(ax2, position = :ct)
    fig
end
```

Left: the two stress distributions lie on top of each other. Right: the centreline deflections coincide, and the top-fibre curve is offset upward by the constant $\nu M c^2/(2EI) \approx 0.01$ mm — far too small to see at this scale.

Stress field

fig_stress =




```
fig_stress = let
    fig = Figure(size = (900, 560))
    fields = [("σₓ", Σx), ("σᵧ", Σy), ("τₓᵧ", Txy)]
    smax = maximum(abs, Σx)
    cr = (-smax, smax)          # symmetric diverging scale: tension (+) vs compression (-)
    for (i, (name, S)) in enumerate(fields)
        lo, hi = round.(extrema(S), digits = 1)
        ax = Axis(fig[i, 1], title = "$name ∈ [$lo, $hi] MPa",
                  xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())
        heatmap!(ax, xs, ys, S, colormap = :RdBu, colorrange = cr)
        i < 3 && hidexdecorations!(ax, grid = false, ticks = false)
    end
    Colorbar(fig[1:3, 2], colormap = :RdBu, limits = cr, label = "stress [MPa]")
    Label(fig[0, :], "Pure bending — stress field from ϕ = −M y³ / (4c³)", fontsize = 18, font = :bold)
    rowgap!(fig.layout, 8)
    fig
end
```

σ_x varies linearly through the depth and is independent of x — the same at every cross-section, exactly as in beam theory; σ_y and τ_{xy} vanish identically.

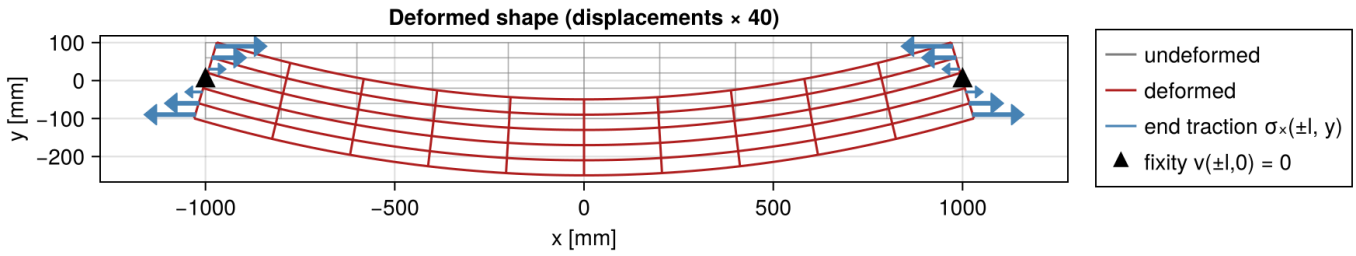
Deformed shape

Displacements are exaggerated by a scale factor. The beam bends into a circular arc of radius $\rho = EI/M$, the centre of the span deflects by $v(0,0) = -Ml^2/(2EI)$, and plane sections remain plane (the vertical grid lines stay straight and rotate).

```
mag = 40
```

```
mag = 40   # displacement magnification for plotting
```

```
fig_deformed =
```




```julia
fig_deformed = let
    fig = Figure(size = (900, 360))
    ax = Axis(fig[1, 1], title = "Deformed shape (displacements × $mag)",
            xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())

    # undeformed grid
    for yi in ys[1:4:end]
        lines!(ax, xs, fill(yi, length(xs)), color = (:gray, 0.6), linewidth = 0.8)
    end
    for xi in xs[1:8:end]
        lines!(ax, fill(xi, length(ys)), ys, color = (:gray, 0.6), linewidth = 0.8)
    end

    # deformed grid
    Xd = [xi + mag * U[i, j] for (i, xi) in enumerate(xs), j in eachindex(ys)]
    Yd = [yj + mag * V[i, j] for i in eachindex(xs), (j, yj) in enumerate(ys)]
    for j in 1:4:length(ys)
        lines!(ax, Xd[:, j], Yd[:, j], color = :firebrick, linewidth = 1.5)
    end
    for i in 1:8:length(xs)
        lines!(ax, Xd[i, :], Yd[i, :], color = :firebrick, linewidth = 1.5)
    end

    # end moments drawn as the linear traction the solution implies on x = ±l
    for (iend, sgn) in ((length(xs), 1.0), (1, -1.0))
        for j in 2:3:length(ys)-1
            s = Σx[iend, j] / maximum(abs, Σx)     # -1 … 1
            arrows2d!(ax, [Xd[iend, j]], [Yd[iend, j]], [sgn * 150 * s], [0.0], color =
            :steelblue)
        end
    end

    # supports
    scatter!(ax, [-p.l, p.l], [0.0, 0.0], marker = :utriangle, markersize = 18, color =
    :black)


```

```
    Legend(fig[1, 2], [LineElement(color = :gray), LineElement(color = :firebrick),
    LineElement(color = :steelblue),
                       MarkerElement(marker = :utriangle, color = :black, markersize =
                   14)],
          ["undeformed", "deformed", "end traction σₓ(±l, y)", "fixity v(±l,0) = 0"])
    fig
```

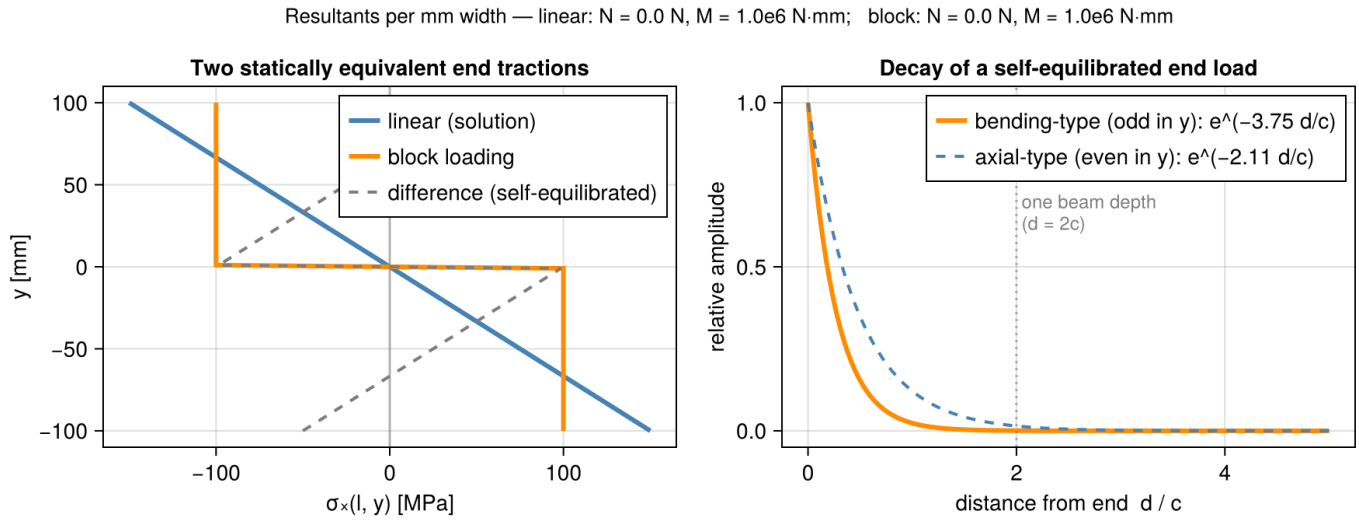
9. What the relaxed end conditions mean numerically

The solution implies the **linear** end traction drawn in blue above. Suppose instead the moment is applied by a pair of bearing blocks that press uniformly on the upper and lower halves of the end face (a block loading):

$$\sigma_x^{\text{block}}(\pm l, y) = \begin{cases} -M/c^2 & 0 < y < c \\ +M/c^2 & -c < y < 0 \end{cases}$$

Both profiles have zero net force and moment M, so **both satisfy (8.1.15)** and the polynomial solution cannot distinguish them. Their difference is a self-equilibrated load. Near the end the real stresses follow whichever profile is actually applied; away from the end the difference decays and both tend to the same $-My/I$.

fig_endloads =




```julia
fig_endloads = let
    fig = Figure(size = (900, 360))
    yy = range(-p.c, p.c, length = 201)
    σ_lin   = [σx_f(p.l, yi, p.M, p.l, p.c, p.E, p.ν) for yi in yy]
    σ_block = [-sign(yi) * p.M / p.c^2 for yi in yy]
    σ_diff  = σ_block .- σ_lin

    ax1 = Axis(fig[1, 1], title = "Two statically equivalent end tractions",
               xlabel = "σₓ(l, y) [MPa]", ylabel = "y [mm]")
    lines!(ax1, σ_lin,   yy, color = :steelblue, linewidth = 3, label = "linear (solution)")
    lines!(ax1, σ_block, yy, color = :darkorange, linewidth = 3, label = "block loading")
    lines!(ax1, σ_diff,  yy, color = :gray, linewidth = 2, linestyle = :dash, label = "difference (self-equilibrated)")
    vlines!(ax1, [0.0], color = (:gray, 0.5))
    axislegend(ax1, position = :rt)

    # resultants of each profile (composite Simpson's rule, exact for both profiles) — identical by construction
    simpson(f) = (h = step(yy); n = length(f); h/3 * (f[1] + f[n] + 4sum(f[2:2:n-1]) + 2sum(f[3:2:n-2])))
    N_lin, N_blk = simpson(σ_lin), simpson(σ_block)
    M_lin, M_blk = -simpson(σ_lin .* yy), -simpson(σ_block .* yy)

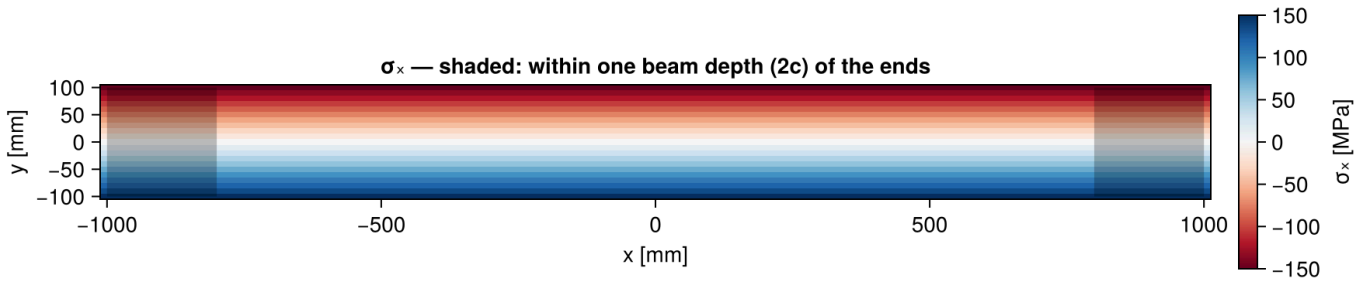
    ax2 = Axis(fig[1, 2], title = "Decay of a self-equilibrated end load",
               xlabel = "distance from end  d / c", ylabel = "relative amplitude")
    d = range(0, 5, length = 200)
    lines!(ax2, d, exp.(-3.75 .* d), color = :darkorange, linewidth = 3,
          label = "bending-type (odd in y): e^(-3.75 d/c)")
    lines!(ax2, d, exp.(-2.11 .* d), color = :steelblue, linewidth = 2, linestyle = :dash,
          label = "axial-type (even in y): e^(-2.11 d/c)")
    vlines!(ax2, [2.0], color = (:gray, 0.6), linestyle = :dot)
    text!(ax2, 2.05, 0.6, text = "one beam depth\n(d = 2c)", fontsize = 12, color = :gray)
    axislegend(ax2, position = :rt)
```

```
31
32      Label(fig[0, :], "Resultants per mm width — linear: N = $(round(N_lin, digits = 3))
        N, M = $(round(M_lin, digits = 1)) N·mm;   block: N = $(round(N_blk, digits = 3))
        N, M = $(round(M_blk, digits = 1)) N·mm",
33          fontsize = 13)
34      fig
35  end
```

The decay rates are the real parts of the leading Papkovich–Fadle eigenvalues for a strip of half-depth c (roots of $\sin 2\lambda \pm 2\lambda = 0$: $\lambda \approx 2.11 + 1.13i$ for loads symmetric in y, $\lambda \approx 3.75 + 1.38i$ for antisymmetric, bending-type loads). They are quoted here as an order-of-magnitude guide, not computed in this notebook. For our beam c = 100.0 mm, so a bending-type self-equilibrated end load has decayed to about $e^{-3.75} \approx 2\%$ at $d = c$ = 100.0 mm and to about $e^{-7.5} \approx 0.06\%$ one beam depth ($d = 2c$ = 200.0 mm) in from each end. Over the remaining $2l - 4c$ = 1600.0 mm the solution is accurate regardless of how the moment is applied.

The shaded regions below mark this zone on the stress plot. Inside them the plotted σ_x is exact only if the moment really is applied as the linear traction; for any other statically equivalent loading the true stresses there differ. Outside them the plot is accurate for every loading with resultants $(0, -M)$.

```
fig_zone =
```




```
1  fig_zone = let
2      fig = Figure(size = (900, 260))
3      ax = Axis(fig[1, 1], title = "σₓ — shaded: within one beam depth (2c) of the ends",
4              xlabel = "x [mm]", ylabel = "y [mm]", aspect = DataAspect())
5      smax = maximum(abs, Σx)
6      heatmap!(ax, xs, ys, Σx, colormap = :RdBu, colorrange = (-smax, smax))
7      poly!(ax, Point2f[(-p.l, -p.c), (-p.l + 2p.c, -p.c), (-p.l + 2p.c, p.c), (-p.l,
       p.c)], color = (:black, 0.25))
8      poly!(ax, Point2f[( p.l, -p.c), ( p.l - 2p.c, -p.c), ( p.l - 2p.c, p.c), ( p.l,
       p.c)], color = (:black, 0.25))
9      Colorbar(fig[1, 2], colormap = :RdBu, limits = (-smax, smax), label = "σₓ [MPa]")
10     fig
11 end
```

10. Summary

quantity	symbolic	numeric
ϕ	$-\frac{M}{4c^3}y^3$	–
I (unit thickness)	$2c^3/3$	666667 mm⁴/mm
σ_x	$-My/I$	$\sigma_x(0,\pm c) = \mp$ 150.0 MPa
σ_y, τ_{xy}	0	0
$u(x,y)$	$-Mxy/(EI)$	$u(l,c) =$ -0.75 mm
$v(x,y)$	$\frac{M}{2EI}[\nu y^2 + x^2 - l^2]$	$v(0,0) =$ -3.75 mm
$v - v^{EB}$	$\frac{\nu M}{2EI}y^2$	$v(0,c) - v^{EB}(0) =$ 0.0112 mm

- A cubic Airy function satisfies the biharmonic equation identically; the pointwise conditions on $y = \pm c$ kill every coefficient but A_{03}, and the *relaxed* (resultant) end conditions fix $A_{03} = -M/(4c^3)$.
- The end conditions are relaxed because the exact end traction is unknown and a pointwise match is not achievable with a polynomial. By Saint-Venant's principle, the price is only paid within about one beam depth of each end, where the solution is not unique and its details depend on how M is actually applied.
- In pure bending the elasticity solution and Euler–Bernoulli beam theory give the **same** bending stress and the same centreline deflection; the elasticity solution adds only the Poisson (anticlastic) term $\nu M y^2/(2EI)$ to v.